University of Zambia
Centre for Information and Communications Technology

TECHNICAL DOCUMENT

Technical Documentation

Equipment Specification & Request Management System

VERSION	1.0
DATE	13 October 2026
PREPARED FOR	CICT developers and system administrators
OWNER	CICT, University of Zambia

Contents

About this document . . . 4

1. System overview . . . 5
   1.1 Purpose . . . 5
   1.2 Scope . . . 5
   1.3 End-to-end process at a glance . . . 5
   1.4 Glossary . . . 6

2. Architecture . . . 7
   2.1 Components . . . 7
   2.2 Design principles . . . 8

3. Backend structure . . . 9
   3.1 Package layout . . . 9
   3.2 Request handling flow . . . 9
   3.3 Error handling . . . 9

4. Security . . . 11
   4.1 Sign-in flow . . . 11
   4.2 Authenticating each API call . . . 11
   4.3 Sign-in rate limiting . . . 11
   4.4 Public and protected endpoints . . . 12
   4.5 Roles . . . 12
   4.6 Request-level access rules . . . 12
   4.7 Other security controls . . . 13

5. Request workflow . . . 14
   5.1 Statuses . . . 14
   5.2 Transitions . . . 15
   5.3 Submission validation . . . 16
   5.4 CICT approval and specification snapshots . . . 17
   5.5 Tier recommendation . . . 17
   5.6 Reference numbers . . . 17

6. Procurement pipeline . . . 18
   6.1 Rules by step . . . 18

6.2 Printable documents . . . 18

7. Notifications . . . 19

7.1 Recipient rules . . . 19

7.2 In-app notifications . . . 19

7.3 Email . . . 19

8. Data model . . . 20

8.1 Tables . . . 20

8.2 Delete behaviour and guards . . . 21

9. REST API reference . . . 22

10. Frontend structure . . . 25

10.1 Source layout . . . 25

10.2 Pages and who sees them . . . 25

10.3 Conventions for contributors . . . 26

11. Configuration . . . 27

11.1 Backend settings . . . 27

11.2 Frontend settings . . . 27

11.3 In-application settings . . . 28

12. Building, running and deploying . . . 29

12.1 Prerequisites . . . 29

12.2 Run locally . . . 29

12.3 Production build . . . 29

12.4 Production checklist . . . 30

13. Testing . . . 31

13.1 Test suites . . . 31

13.2 Running the tests . . . 31

13.3 Frontend checks . . . 32

14. Operations . . . 33

14.1 Backup and restore . . . 33

14.2 Monitoring and logs . . . 33

14.3 Troubleshooting . . . 33

14.4 Making schema changes . . . 34

15. Known limitations and recommendations 35

Appendix A — Lookup lists 36

Appendix B — Equipment categories 37

Appendix C — System settings 38

Appendix D — Database migrations 39

About this document

This document describes how the UNZA CICT Equipment Specification & Request Management System is built, configured, secured, deployed, tested and operated. It is written for the CICT developers who maintain the code and the system administrators who run it.

How to use this document

- Read **Sections 1 to 3** first for the big picture: what the system does, how its parts fit together and the technology used.
- **Sections 4 to 9** explain the internals: security, the request workflow, procurement, notifications, the data model and the REST API.
- **Sections 10 to 13** are practical: frontend structure, configuration, building and deploying, and testing.
- **Sections 14 and 15** cover day-to-day operations, troubleshooting and the known limitations to address before production.
- The appendices list reference data: lookup lists, equipment categories, system settings and the database migration history.

Document control

Document	Technical Documentation — UNZA CICT Equipment Specification & Request Management System
Version	1.0 (first issue)
Date	13 October 2026
Owner	Centre for Information and Communications Technology (CICT), University of Zambia
Code base	Frontend: repository root (TanStack Start, React, TypeScript). Backend: `backend/` (Spring Boot, Java 17).
Companion document	User Manual (same system), which explains the system from each user's point of view.

Conventions

- `Monospace text` marks code, file paths, configuration keys, API paths and values typed exactly as shown.
- Request statuses are written as they are stored, for example `pending_manager`; the user-facing label is given where it differs ("Awaiting manager").
- Paths such as `service/RequestService.java` are relative to `backend/src/main/java/zm/unza/cict/specification/` unless stated otherwise.

1. System overview

The system gives the University of Zambia one controlled, traceable route for acquiring ICT equipment: from a staff member's request, through management and CICT technical approval, to procurement, receipt into stores and installation verification.

1.1 Purpose

Before this system, equipment requests reached CICT through email and paper forms, making it hard to apply consistent technical standards, track progress or prove who approved what. The system addresses this by:

- **Standardising specifications.** Every item is matched to a CICT-approved *specification profile* (Standard, Enhanced or Specialised tier) for its equipment category.
- **Enforcing an approval chain.** A request must be approved by the requester's manager (the business need) and then by CICT (the technical specification) before procurement can act on it.
- **Tracking delivery.** Procurement, Stores and CICT Support record quotations, evaluations, purchases, receipts and installation checks against the same request.
- **Keeping a full audit trail.** Every status change is logged as a request event, and administrative changes are written to a system audit log.
- **Producing official documents.** Printable request forms, specification sheets, purchase orders, stores receipts and verification certificates are generated from the recorded data.

1.2 Scope

In scope: staff sign-in through the UNZA HR system; request creation and approval; CICT specification review; the procurement pipeline through to verification; notifications; reporting; administration of roles, organisational units, catalogue, lookup lists and settings; auditing.

Out of scope: supplier management, financial commitment and payment (these remain in University procurement and finance systems); asset register management after verification.

1.3 End-to-end process at a glance

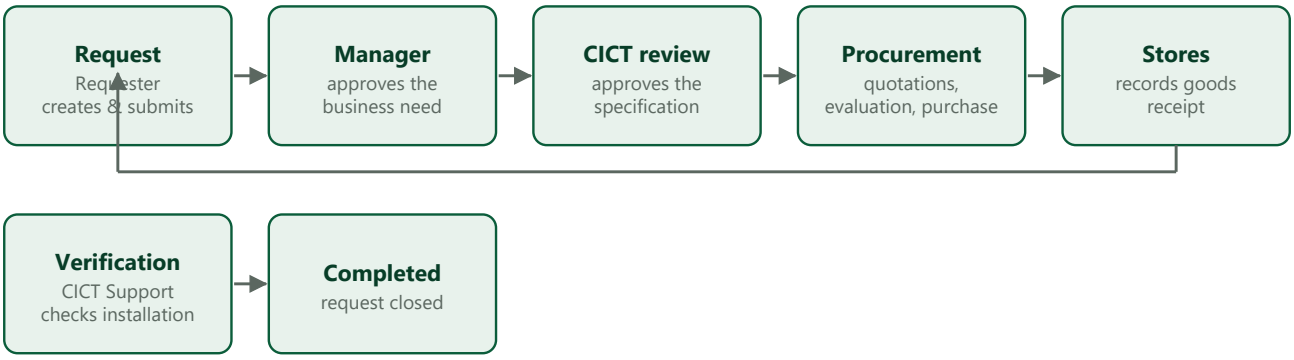


Figure 1 — The request journey. Each box is a stage owned by a different role.

1.4 Glossary

Term	Meaning
Request	A staff member's application for one or more ICT equipment items, identified by a reference number such as `202610130001`.
Item	One line of a request: an equipment category, quantity, purpose, users and location.
Equipment category	A class of equipment in the catalogue (for example Laptop, Printer, Network Switch).
Specification profile	A CICT-approved minimum specification for a category at a given tier, listing technical attributes.
Tier	The level of a profile: Standard, Enhanced or Specialised.
Position level	The staff member's grade from HR (for example Lecturer, Dean). Used to recommend a tier.
Delegation	A manager temporarily granting another staff member authority to approve on their behalf.
Request event	An entry in a request's audit trail recording an action, its actor, the status change and any note.
Audit log	The system-wide log of administrative and procurement actions (separate from request events).
HR system	The University's HR API (`hr.unza.zm`) used for sign-in and staff profile data.
JWT	JSON Web Token: the signed session token the backend issues after sign-in.

2. Architecture

The system is a classic three-tier web application: a React single-page frontend, a stateless Spring Boot REST API, and a PostgreSQL database, with two external integrations.

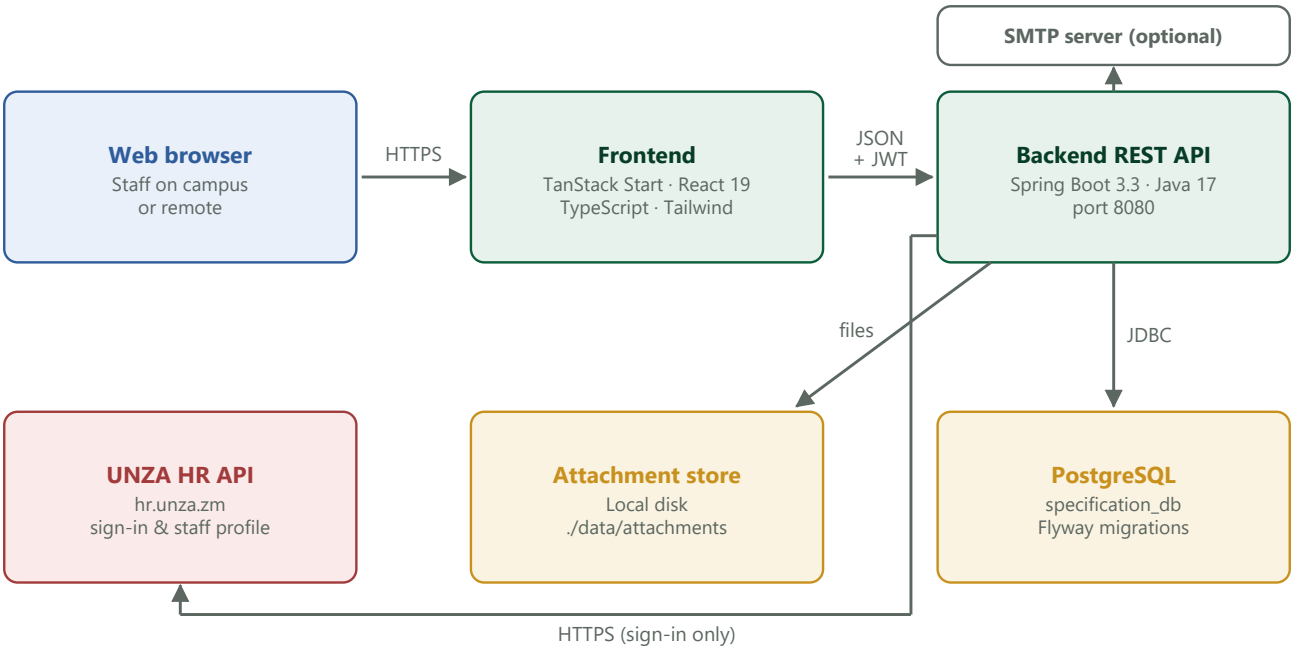


Figure 2 — Deployment components and how they communicate.

2.1 Components

Component	Responsibility	Technology
Frontend	All screens; form validation; role-aware navigation; printable documents; calls the API with the user's token.	TanStack Start 1.168 (React 19, TanStack Router, React Query 5), TypeScript 5.8, Tailwind CSS 4, Radix UI / shadcn components, Vite 8; built by Nitro for Cloudflare Workers
Backend API	Business rules and the workflow state machine; authorisation; persistence; file storage; HR integration; notifications and email.	Spring Boot 3.3.5 on Java 17 (runs on 21): Web, Data JPA, Security, Validation, Mail, Actuator; jjwt 0.12.6; springdoc-openapi 2.6
Database	All persistent data except attachment file contents.	PostgreSQL (16+; developed on 18). Schema managed by Flyway (V1–V16).
Attachment store	Uploaded files (supporting documents, vendor quotations).	Local filesystem directory, configurable via `ATTACHMENTS_DIR`.
UNZA HR API	Authenticates staff credentials and returns the staff profile.	External REST API (two calls: login, then metadata).
SMTP (optional)	Email copies of notifications.	Any SMTP server with STARTTLS; disabled by default.

2.2 Design principles

- **The backend is the authority.** The frontend hides actions a user cannot take, but every rule (who may act, in which status, with which data) is enforced again by the API. Never rely on the UI for security.
- **Stateless API.** No server sessions. Each request carries a JWT; the user's roles are re-read from the database on every call, so role changes and deletions take effect immediately.
- **Schema by migration.** Hibernate only *validates* the schema (`ddl-auto: validate`). All schema changes are versioned Flyway scripts.
- **Everything traceable.** State changes write a request event; administrative and procurement actions write the audit log.
- **Fail clearly.** All errors return one JSON shape with a human-readable message that the frontend shows directly to the user.

3. Backend structure

The backend follows a conventional layered design: controllers handle HTTP, services hold business rules and transactions, repositories access the database, and DTOs define the API contract.

3.1 Package layout

Root package: `zm.unza.cict.specification`.

Package	Contents and role
`controller`	13 REST controllers, one per resource area (requests, procurement, attachments, users, org units, categories, spec profiles, lookups, settings, delegations, notifications, audit log, auth). Thin: they read the current user and delegate to services.
`service`	Business logic and transaction boundaries (`@Transactional`). Key classes: `RequestService` (workflow), `ProcurementService` (pipeline), `NotificationService`, `AuthService`, `UserService`, `ReferenceNumberService`, `RequestAccessService` (view/edit rules), `AuditLogService`, `EmailService`.
`repository`	Spring Data JPA interfaces, one per entity, including custom queries (for example request visibility and delegation look-ups).
`domain`	18 JPA entities mapped to the database tables (Request, RequestItem, User, SpecProfile, and so on). Lombok generates getters, setters and builders.
`dto`	Java records for requests and responses, grouped by area (`request`, `procurement`, `user`, `catalogue`, `lookup`, `orgunit`, `delegation`, `notification`, `audit`, `auth`). Bean Validation annotations (`@NotBlank`, `@Positive` …) live here.
`security`	`JwtService` (issue and parse tokens), `JwtAuthenticationFilter` (per-request authentication), `AuthenticatedUser` (role helpers), `CurrentUser` (access to the signed-in user), `LoginRateLimiter`.
`hr`	`HrClient` and `HrStaff`: the UNZA HR integration.
`config`	`SecurityConfig` (filter chain, CORS) and `AppProperties` (typed `app.*` configuration).
`common`	Enums (`AppRole`, `RequestStatus` with JPA converters) and exceptions with the `GlobalExceptionHandler`.

3.2 Request handling flow

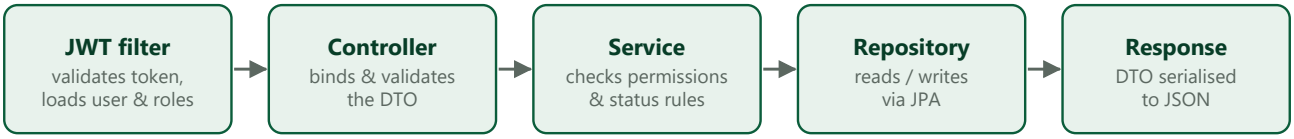


Figure 3 — How one API call passes through the backend layers.

3.3 Error handling

`GlobalExceptionHandler` converts every exception into the same JSON body, so the frontend can always show `message` to the user and highlight fields from `fieldErrors`:

```
{
  "timestamp": "2026-10-13T07:27:04.345Z",
  "status": 400,
  "error": "Bad Request",
  "message": "This category is used by existing requests. Retire it instead of deleting it.",
  "fieldErrors": null
}
```

Exception	HTTP status	Typical cause
`BadRequestException`, `IllegalArgumentException`	400	A business rule was broken (wrong status, missing reason, invalid value).
`MethodArgumentNotValidException`, `ConstraintViolationException`	400	DTO validation failed; `fieldErrors` maps each field to its message.
`BadCredentialsException`	401	Invalid credentials.
`ForbiddenException`, `AccessDeniedException`	403	The user lacks the role or relationship to the request.
`NotFoundException`	404	The record does not exist (or was deleted).
`DataIntegrityViolationException`	409	A database constraint blocked the change; the record is still in use.
`MaxUploadSizeExceededException`	413	Upload larger than 25 MB.
`TooManyRequestsException`	429	Too many failed sign-in attempts (see 4.3).
`HrIntegrationException`	502	The HR system rejected the credentials, was unreachable or returned an error.
Any other exception	500	Unexpected error; the stack trace is logged and a generic message returned.

4. Security

Users never have passwords in this system. Identity comes from the UNZA HR system; authorisation comes from roles stored in the system's own database.

4.1 Sign-in flow

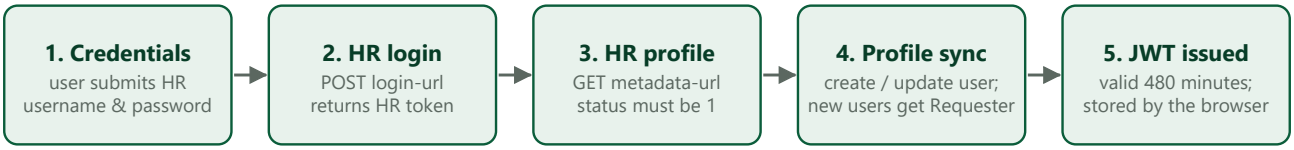


Figure 4 — Sign-in through the UNZA HR system.

1. The frontend posts `{identifier, password}` to `POST /api/auth/login` (public endpoint).
2. `HrClient` calls the HR login URL with the credentials and receives an HR access token. A 401 or 403 from HR becomes "Those staff credentials were not accepted by the HR system."
3. `HrClient` calls the HR metadata URL with that token. Only staff whose HR `status` is `1` (active) may continue; others receive "Your staff account is not active. Please contact HR."
4. `AuthService` finds the local user by staff (man) number, falling back to email, creating one if needed. Name, job title, position level (from the HR position hierarchy, title-cased) and phone are copied from HR on **every** sign-in, and `hr_synced_at` is stamped. Brand-new users are given the `requester` role.
5. `JwtService` issues an HS256-signed token containing the user id (`sub`), email and roles, valid for `app.jwt.expiration-minutes` (480 = 8 hours). The response also contains the user profile.

> **Note.** Because HR details are overwritten at each sign-in, corrections to a person's name, job title or position level must be made in HR. The system's copy refreshes the next time they sign in.

4.2 Authenticating each API call

`JwtAuthenticationFilter` runs before every request. It verifies the token's signature and expiry, then loads the user **and their current roles from the database** (the roles inside the token are ignored). If the user no longer exists the request is treated as anonymous. This means an administrator's role change or account deletion takes effect on the user's very next request, without waiting for the token to expire.

When the API returns 401, the frontend clears the stored token and sends the user back to the sign-in page.

4.3 Sign-in rate limiting

`LoginRateLimiter` counts failed sign-in attempts per identifier. After **5 failures** the identifier is locked for **15 minutes**, and further attempts return HTTP 429 with the number of minutes remaining. A successful sign-in clears the counter. The counter is held in memory, so it resets when the backend restarts and is not shared between multiple backend instances.

4.4 Public and protected endpoints

Endpoint pattern	Access	Reason
`/api/auth/**`	Public	Sign-in must work before a token exists.
`GET /api/settings`	Public	Branding, support email, declaration text and SLA days are shown on the sign-in page.
`/actuator/health`	Public	Load balancer and monitoring health check (no details exposed).
`/swagger-ui/**`, `/v3/api-docs/**`	Public	Interactive API documentation. Consider restricting in production (see Section 15).
Everything else	Authenticated	A valid JWT is required; fine-grained checks happen in the services.

4.5 Roles

Roles are stored in `user_roles` (one row per user per role). A user can hold any combination. `AuthenticatedUser` defines two composite checks used throughout the code:

- **isCict** = `cict_reviewer` or `cict_manager` or `admin`.
- **isOversight** = isCict, or `procurement`, `auditor`, `stores` or `cict_support`.

Role (stored value)	What it permits
Requester (`requester`)	Create, edit, submit, withdraw and track own requests; respond to clarification. Given automatically to every new user.
Manager (`manager`)	Shows the Manager approvals queue. Approval rights come from being the *nominated* manager on a request (or their active delegate), not from the role alone.
CICT reviewer (`cict_reviewer`)	Technical review: approve specification, request clarification, reject. Maintain specification profiles. Oversight views.
CICT manager (`cict_manager`)	Same permissions as CICT reviewer.
Procurement (`procurement`)	Add and evaluate quotations, record purchases, restart procurement after a failed verification. Oversight views.
Stores (`stores`)	Record stores (goods) receipts. Oversight views.
CICT Support (`cict_support`)	Record installation verification (pass or needs return). Oversight views.
Auditor (`auditor`)	Read-only oversight: all requests, reports, directory and the audit log.
Administrator (`admin`)	Everything: all CICT permissions, every pipeline action, and the Administration area (roles, units, categories, accounts, lists, settings, audit log).

4.6 Request-level access rules

`RequestAccessService` decides who may see or change an individual request:

Rule	Who
View a request	Oversight users (all requests); the requester; the person who submitted it; the nominated manager; anyone currently holding the manager's delegation.

Rule	Who
Edit a draft	CICT users; the requester or submitter; the nominated manager or their delegate. Only requests in `draft` status can be edited.
Delete a request	Only the person who submitted it, and only while it is a draft.
Withdraw	The requester or submitter, while the request is `draft`, `pending_manager`, `pending_cict` or `needs_clarification`.
Delete an attachment	The person who uploaded it, or any CICT user.

4.7 Other security controls

- **CORS:** only origins listed in `app.cors.allowed-origins` may call the API from a browser.
- **CSRF:** disabled, which is appropriate because the API uses bearer tokens rather than cookies.
- **File uploads:** limited to 25 MB and to the extensions pdf, doc, docx, xls, xlsx, ppt, pptx, txt, csv, jpg, jpeg, png, gif, webp and heic. Stored file names are generated, and paths are normalised to stop path traversal outside the attachments directory.
- **Profile self-service:** `PATCH /api/users/me` cannot change position level or organisational unit (except for administrators), because those fields drive approval routing and specification tiers.
- **Deletion guards:** equipment categories and specification profiles referenced by existing requests cannot be deleted; they must be retired instead (see 8.3).
- **Secrets:** the JWT secret, database password and SMTP password must be supplied through environment variables in production. The backend logs a prominent warning at start-up if the default JWT secret is still in use.

> **Important.** The default JWT secret is published in the source code. Anyone who knows it can forge a token for any user. Set `JWT_SECRET` to a long random value (at least 32 bytes) on every deployed server.

5. Request workflow

The heart of the system is a state machine on the request's `status` column. Only specific actions, by specific people, can move a request from one status to another.

5.1 Statuses

Status (stored)	Label shown to users	Meaning
`draft`	Draft	Being prepared, or returned by the manager for correction. Visible only to the requester side; editable.
`pending_manager`	Awaiting manager	Submitted; waiting for the nominated manager (or delegate).
`pending_cict`	With CICT	Business need approved; waiting for CICT technical review.
`needs_clarification`	Clarification needed	CICT has asked the requester a question.
`approved`	CICT approved	Specification approved. Procurement can start. Not an authority to spend.
`procurement`	With procurement	At least one quotation has been recorded.
`purchased`	Purchased	A purchase has been recorded against a passing quotation.
`in_stores`	In stores	Goods received by Stores; awaiting CICT Support verification.
`completed`	Completed	Verified as installed and working. Final.
`return_required`	Return required	Verification failed; goods must be returned and procurement restarted.
`rejected`	Rejected	Refused by the manager or CICT. Final.
`withdrawn`	Withdrawn	Cancelled by the requester. Final.

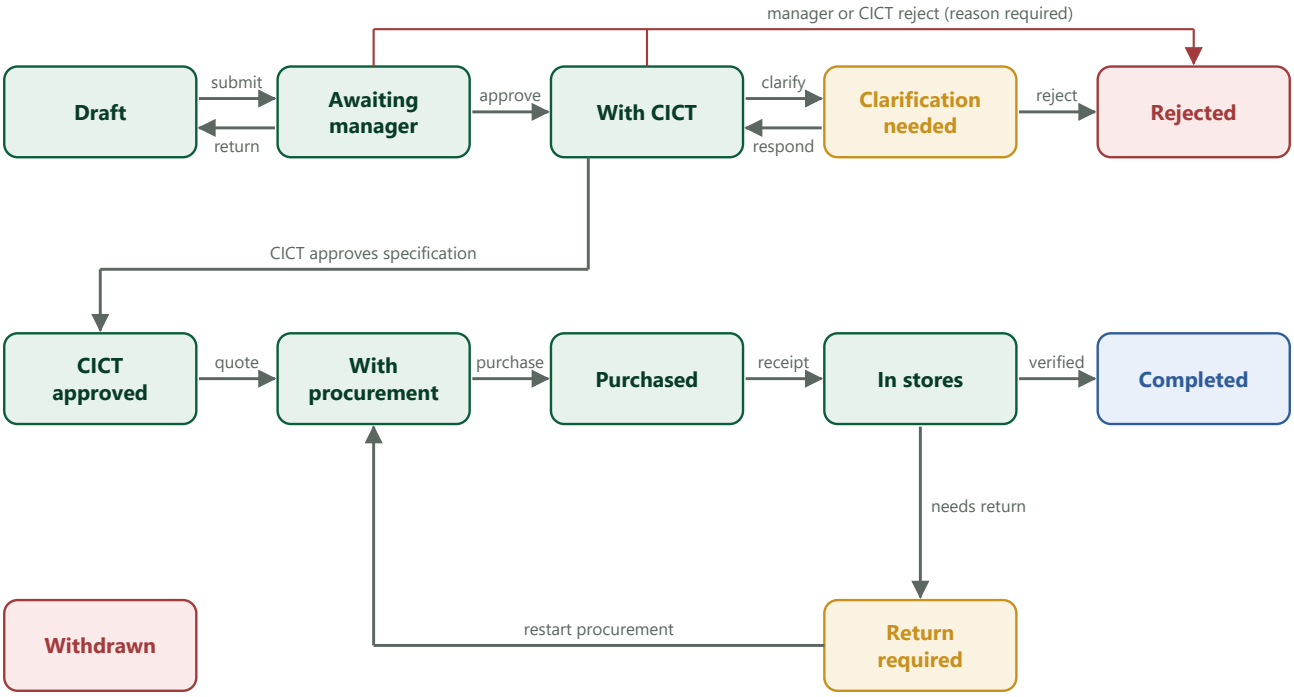


Figure 5 — Request status transitions. Red boxes are final refusals; blue is the successful end.

5.2 Transitions

From	Action (API)	Who may act	To	Reason?
(new)	Save draft — `POST /api/requests?submit=false`	Any signed-in user	`draft`	No
(new) / `draft`	Submit — `POST /api/requests?submit=true`, `PUT /api/requests/{id}?submit=true` or `POST …/{id}/submit`	Requester side (edit rights)	`pending_manager`	No
`pending_manager`	`manager-decision` approved	Nominated manager or active delegate	`pending_cict`	Optional
`pending_manager`	`manager-decision` returned	Manager or delegate	`draft`	**Yes**
`pending_manager`	`manager-decision` rejected	Manager or delegate	`rejected`	**Yes**
`pending_cict` / `needs_clarification`	`cict-decision` approved	CICT (isCict)	`approved`	Optional
`pending_cict` / `needs_clarification`	`cict-decision` needs_clarification	CICT	`needs_clarification`	**Yes**
`pending_cict` / `needs_clarification`	`cict-decision` rejected	CICT	`rejected`	**Yes**
`needs_clarification`	`respond-clarification`	Requester or submitter	`pending_cict`	**Yes**

From	Action (API)	Who may act	To	Reason?
`draft`, `pending_manager`, `pending_cict`, `needs_clarification`	`withdraw`	Requester or submitter	`withdrawn`	No
`approved`	First quotation added	Procurement / admin	`procurement`	—
`procurement`	Purchase recorded	Procurement / admin	`purchased`	—
`purchased`	Stores receipt recorded	Stores / admin	`in_stores`	—
`in_stores`	Verification pass	CICT Support / admin	`completed`	No
`in_stores`	Verification needs_return	CICT Support / admin	`return_required`	**Yes**
`return_required`	`restart-procurement`	Procurement / admin	`procurement`	—

Every transition writes a `request_events` row (actor, action text, from-status, to-status, note) and calls `NotificationService.notifyStatusChange` (Section 7).

5.3 Submission validation

A request can be saved as a draft with almost nothing filled in, but `validateForSubmission` enforces the following before it leaves the draft stage. Each failure returns a 400 with the message shown:

Check	Message returned
Title present	Add a descriptive request title.
Unit / School selected	Select the requesting Unit or School.
Department entered	Enter the requesting department.
Designation of intended user	Enter the designation of the intended user.
Request type selected	Select the request type.
Business context selected	Select the business context.
Justification at least 10 characters	Give a business justification.
At least one item	Add at least one equipment item.
Each item has a category	Item N needs an equipment category.
Each item quantity ≥ 1	Item N quantity must be one or more.
Each item has a purpose	Describe what item N will be used for.
Printer items state number of users	Specify the number of users for item N.
Software items have a category (Design, Teaching, Development, Other)	Select a valid software category for item N.
Approving manager chosen	Choose the approving manager.
Declaration accepted	Accept the requester declaration before submitting.

5.4 CICT approval and specification snapshots

When CICT approves, the request may carry `itemDecisions`, one per item, choosing an `approvedProfileId`, an optional exception justification and a reviewer note. For each item the backend:

1. Uses the chosen profile, or falls back to the item's recommended profile.
2. Rejects the approval if the profile belongs to a different category ("The approved specification does not match its equipment category.") or has been retired.
3. Stores the profile link **and a snapshot** (`approved_spec_snapshot`: name, tier and specs as JSON). The snapshot means later edits to the profile never change what was approved for this request.
4. Finally checks that every item has an approved profile or snapshot; otherwise: "Choose an approved technical specification for item N before approval."

5.5 Tier recommendation

When an item is added, the system suggests a specification profile. The same algorithm runs in the frontend (`recommendTier` in `src/lib/domain.ts`) for instant feedback and in the backend (`SpecProfileService.recommend`) as the stored value:

1. Take the active profiles for the item's category.
2. Prefer a profile whose "suits position levels" list contains the intended user's position level (case- and space-insensitive). The level comes from the item's designation, otherwise the requester's HR position level.
3. Otherwise fall back to the lowest tier available (Standard, then Enhanced, then Specialised).
4. For shared equipment, escalate by number of users: more than 5 users needs at least Enhanced; more than 15 needs Specialised.

5.6 Reference numbers

Document	Format	Example	How it is generated
Request	`yyyyMMdd` + 4-digit daily sequence	`202610130001`	Upsert counter in `request_number_sequences`, one row per day; resets daily.
Quotation evaluation	`UNZA/CICT/EVAL/yyyy/nnnn`	`UNZA/CICT/EVAL/2026/0003`	Upsert counter in `reference_sequences` keyed by (prefix, year).
Verification certificate	`UNZA/CICT/VERIFY/yyyy/nnnn`	`UNZA/CICT/VERIFY/2026/0002`	Same counter table, prefix `VERIFY`; resets yearly.

The counters use a single `INSERT … ON CONFLICT DO UPDATE` statement, so concurrent requests can never receive the same number.

6. Procurement pipeline

After CICT approval, four roles complete the request. All actions are implemented in `ProcurementService` and exposed under `/api/requests/{requestId}/…`. Administrators may perform any of them.

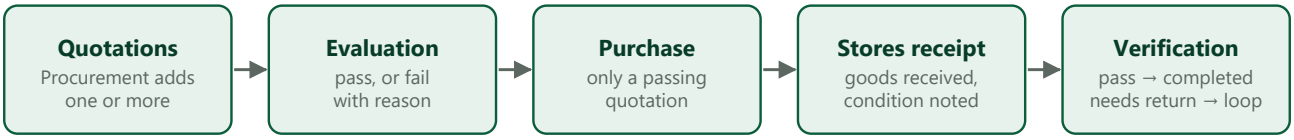


Figure 6 — The procurement pipeline.

6.1 Rules by step

Step	Role	Allowed when	Rules and effects
Add quotation	procurement	`approved` or `procurement`	Vendor name and positive amount required; currency, notes and an optional quote document (an attachment of the same request). The first quotation moves the request to `procurement`.
Evaluate quotation	procurement	`procurement`	Result `pass` or `fail`; a fail needs notes. Issues a `UNZA/CICT/EVAL/…` reference.
Record purchase	procurement	`procurement`	Must reference a quotation with a passing evaluation ("Only a quotation with a passing evaluation can be purchased."). PO number, vendor, amount, currency, notes. Moves to `purchased`.
Stores receipt	stores	`purchased`	Optional condition notes. Moves to `in_stores`.
Verification	cict_support	`in_stores`	Result `pass` → `completed`; `needs_return` (notes required) → `return_required`. Issues a `UNZA/CICT/VERIFY/…` reference.
Restart procurement	procurement	`return_required`	Moves back to `procurement` so a new quotation and purchase can be recorded.

Each step writes a request event and an audit-log entry, and triggers notifications.

6.2 Printable documents

Document	Route	Available from status
Printable request form	`/requests/{id}/print`	Any
Approved specification sheet	`/requests/{id}/spec-sheet`	`approved` onwards
Purchase order	`/requests/{id}/purchase-order`	`purchased` onwards
Stores receipt	`/requests/{id}/stores-receipt`	`in_stores` onwards
Verification certificate	`/requests/{id}/verification-certificate`	`completed` or `return_required`

These are frontend pages rendered on the University letterhead (`Letterhead` component) with a Print button that calls the browser's print dialog; users can print to paper or save as PDF.

7. Notifications

Whenever a request changes status, the system tells the people who need to act next. Notifications are always stored in-app; email copies are optional.

7.1 Recipient rules

New status	Who is notified
`pending_manager`	The nominated manager, plus everyone holding an active delegation from that manager.
`pending_cict`	Every user with the `cict_reviewer` or `cict_manager` role.
Any other status	The requester and the person who submitted the request (if different).

Two filters always apply: duplicates are removed, and **the person who performed the action is never notified about their own action**. The title has the form `<reference> - <status>`, and the body includes the request title.

7.2 In-app notifications

- Stored in the `notifications` table, linked to the user and the request.
- The bell icon in the top bar checks `GET /api/notifications/unread-count` every minute and shows the number.
- Users can mark one or all notifications as read and dismiss (delete) individual notices. Only the owner can read, mark or delete a notification (others receive 403).

7.3 Email

Email is **off by default**. To enable it, set `MAIL_ENABLED=true` and the `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD` and `MAIL_FROM` variables. Each in-app notification is then also emailed to the recipient. The mail health indicator is disabled so an unreachable SMTP server never marks the application unhealthy.

8. Data model

The database has 22 tables. Primary keys are UUIDs; timestamps are stored in UTC. The `requests` table is the hub that most other tables hang from.

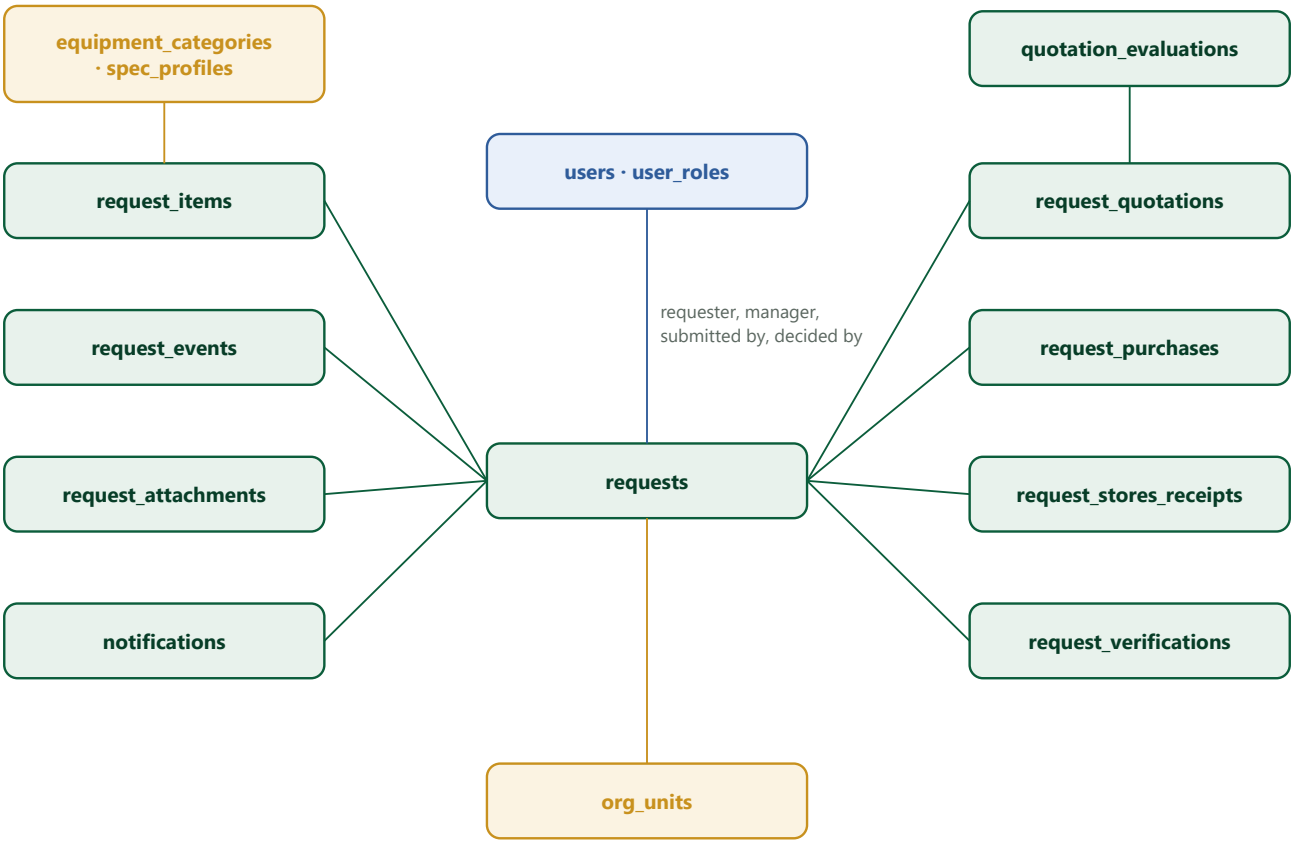


Figure 7 — Main tables. Green tables belong to a request and are deleted with it; blue is people; gold is reference data.

8.1 Tables

Table	Holds
`requests`	One row per request: reference, title, status, requester, submitted-by, on-behalf-of, manager, org unit, department, designation, request type, business context, justification, impact, replacement details, priority, required-by date, budget status, funding source, cost centre, declaration flag, decision timestamps and notes.
`request_items`	Items: category, quantity, purpose, number of users, intended users, location, designation, required software, software category, proposed specification, recommended profile, approved profile, approved spec snapshot (JSON), exception justification, reviewer note, sort order.
`request_events`	The per-request audit trail: actor, action text, from- and to-status, note, timestamp.
`request_attachments`	Metadata of uploaded files (original name, generated storage path, content type, size, kind, uploader). File contents live on disk.
`request_quotations` / `quotation_evaluations`	Vendor quotations (vendor, amount, currency, notes, optional quote attachment) and their pass/fail evaluations with EVAL references.

Table	Holds
`request_purchases`	Purchase records: quotation, PO number, vendor, amount, currency, notes.
`request_stores_receipts`	Goods receipt records with condition notes.
`request_verifications`	Installation verification: result, notes, VERIFY reference.
`users` / `user_roles`	Staff profiles mirrored from HR (email, full name, staff number, job title, position level, phone, campus, org unit, HR sync time) and their role assignments.
`delegations`	Manager-to-delegate authority with reason, start and optional end time, and active flag.
`notifications`	In-app notices per user, with read time.
`org_units`	Schools, directorates, departments, units and centres with code, type, campus, optional parent and active flag.
`equipment_categories`	The catalogue: code, name, description, asset class, lifespan, warranty, indicative unit cost, procurement lead time, review/network/licence flags, eligible position levels, support notes, policy reference, sort order, active flag.
`spec_profiles`	Specification profiles: category, tier, name, summary, specs (JSON key/value), suitable position levels, version, active flag.
`lookup_values`	Configurable drop-down lists keyed by `list_key` (see Appendix A).
`app_settings`	Key/value system settings (see Appendix C).
`audit_log`	System audit trail: entity type, entity id, action, details, actor, timestamp.
`reference_sequences`, `request_number_sequences`, `request_reference_sequences`	Counters for reference numbers (the last is a legacy table from an earlier numbering scheme).

8.2 Delete behaviour and guards

Foreign keys define what happens when a parent row is deleted:

- Deleting a **request** cascades to its items, events, attachments (database rows), notifications, quotations, purchases, receipts and verifications.
- Deleting an **org unit** sets the unit to empty on users, requests and child units.
- Deleting an **equipment category** would cascade-delete its specification profiles and empty the category on request items; deleting a **profile** would empty the recommended and approved profile on items.

> **Important.** Because those last cascades would silently damage the history of approved requests, the services refuse to delete a category used by any request item or still holding profiles, and refuse to delete a profile referenced by any request item. Users are told to **retire** the record instead (set it inactive), which hides it from new requests but keeps history intact.

Deleting a **user** (administrators only, never their own account) removes their account and role assignments. The confirmation in the UI warns that requests they raised are also removed.

9. REST API reference

All endpoints are under `/api`, exchange JSON (except uploads and downloads), and require `Authorization: Bearer <token>` unless marked public. Interactive documentation is served at `/swagger-ui.html` and the OpenAPI document at `/v3/api-docs`.

Authentication

Method	Path	Who	Purpose
POST	/api/auth/login	Public	Sign in with HR credentials; returns {token, user}.
GET	/api/auth/me	Any	The signed-in user's profile and roles.

Requests

Method	Path	Who	Purpose
GET	/api/requests	Any	Requests visible to the caller (see 4.6), including delegated ones.
POST	/api/requests?submit=	Any	Create a request; submit=true validates and submits immediately.
GET	/api/requests/{id}	Viewer	Full request with items.
PUT	/api/requests/{id}?submit=	Editor	Update a draft; optionally submit.
DELETE	/api/requests/{id}	Submitter	Delete a draft.
POST	/api/requests/{id}/submit	Editor	Submit a draft.
POST	/api/requests/{id}/withdraw	Requester	Withdraw.
POST	/api/requests/{id}/manager-decision	Manager / delegate	{decision: approved|returned|rejected, note}.
POST	/api/requests/{id}/cict-decision	CICT	{decision: approved|needs_clarification|rejected, note, itemDecisions[]}.
POST	/api/requests/{id}/respond-clarification	Requester	{action, comment} — the comment is the answer.
GET	/api/requests/{id}/events	Viewer	The request's audit trail.

Attachments

Method	Path	Who	Purpose
GET	/api/requests/{requestId}/attachments	Viewer	List files.
POST	/api/requests/{requestId}/attachments	Viewer	Multipart upload: file, optional kind.
GET	/api/attachments/{id}/download	Viewer	Download with the original file name.
DELETE	/api/attachments/{id}	Uploader / CICT	Delete file and record.

Procurement pipeline

Method	Path	Who	Purpose
GET/POST	/api/requests/{requestId}/quotations	Viewer / procurement	List or add quotations.
POST	/api/requests/{requestId}/quotations/{quotationId}/evaluate	Procurement	{result: pass|fail, notes}.
GET/POST	/api/requests/{requestId}/purchase	Viewer / procurement	Read or record the purchase.
GET/POST	/api/requests/{requestId}/stores-receipt	Viewer / stores	Read or record the receipt.
GET/POST	/api/requests/{requestId}/verification	Viewer / CICT Support	{result: pass|needs_return, notes}.
POST	/api/requests/{requestId}/restart-procurement	Procurement	After a failed verification.

Users and delegation

Method	Path	Who	Purpose
GET	/api/users	Any	All users (used to pick managers and delegates).
GET	/api/users/{id}	Any	One user.
PATCH	/api/users/me	Any	Update own profile fields (not position level or unit unless admin).
DELETE	/api/users/{id}	Admin	Delete a user (not yourself).
GET	/api/users/roles	Any	Own role assignments; oversight users see all.
POST	/api/users/roles	Admin	Assign a role.
DELETE	/api/users/roles/{assignmentId}	Admin	Remove a role.
POST	/api/users/staff-accounts	Admin	Manually provision a staff account.
GET/POST	/api/delegations	Any	List own delegations / delegate authority.
POST	/api/delegations/{id}/deactivate	Delegator	End a delegation early.

Catalogue and reference data

Method	Path	Who	Purpose
GET/POST	/api/equipment-categories	Any / admin	List / create categories.
PUT/DELETE	/api/equipment-categories/{id}	Admin	Update (including retire) / delete if unused.
GET/POST	/api/spec-profiles	Any / CICT	List / create profiles.
PUT/DELETE	/api/spec-profiles/{id}	CICT	Update (including retire) / delete if unused.
GET/POST	/api/org-units	Any / admin	List / create units.

Method	Path	Who	Purpose
PUT	/api/org-units/{id}	Admin	Edit a unit.
PATCH	/api/org-units/{id}/active?active=	Admin	Retire or restore.
GET/POST	/api/lookup-values	Any / admin	List (?includeInactive=) / add list entries.
PUT/DELETE	/api/lookup-values/{id}	Admin	Edit / delete list entries.
GET	/api/settings	Public	All settings.
PUT	/api/settings/{key}	Admin	Change a setting value.

Notifications and audit

Method	Path	Who	Purpose
GET	/api/notifications	Any	Own notifications, newest first.
GET	/api/notifications/unread-count	Any	{count}.
POST	/api/notifications/{id}/read	Owner	Mark one read.
POST	/api/notifications/read-all	Any	Mark all own notifications read.
DELETE	/api/notifications/{id}	Owner	Dismiss.
GET	/api/audit-log?limit=	Auditor / admin	Recent audit entries.
DELETE	/api/audit-log	Admin	Clear the audit log.

10. Frontend structure

The frontend is a TanStack Start application (React 19, TypeScript) generated with Lovable and kept in the repository root. Routing is file-based; data is fetched with React Query.

10.1 Source layout

Path	Contents
`src/routes/`	One file per page. `_authenticated/` holds every signed-in page; its `route.tsx` redirects to `/auth` when no valid session exists and wraps pages in the `AppShell` layout. `routeTree.gen.ts` is generated by the router plugin — do not edit it by hand.
`src/components/`	`AppShell` (sidebar, top bar, mobile menu), `ProcurementPipeline`, `RequestAttachments`, `DelegationPanel`, `NotificationsBell`, `Letterhead`, `StatusBadge`, `QueryError`, admin panels, and `ui/` (shadcn/Radix primitives).
`src/lib/api-client.ts`	`apiGet/Post/Put/Patch/Delete/Upload/Download`: adds the bearer token, parses the error body into `ApiError(status, message, fieldErrors)`, clears the token on 401.
`src/lib/api-types.ts`	TypeScript types mirroring the backend DTOs.
`src/lib/domain.ts`	Status labels, priority options, `recommendTier`, date and money formatting.
`src/lib/session.ts`	Token storage and expiry check.
`src/hooks/useAuth.ts`	Current user, roles and helpers: `hasRole`, `isCict`, `isOversight` (mirroring the backend definitions).
`src/hooks/useLookups.ts`	Cached lookup lists and settings.

10.2 Pages and who sees them

Route	Page	Shown to
`/`, `/help`, `/auth`	Landing page, guidance and policy, sign-in	Everyone (public)
`/dashboard`	Personal summary and queues	All signed-in users
`/requests`, `/requests/new`, `/requests/{id}`, `/requests/{id}/edit`	Request list, form, detail, edit draft	All signed-in users (data filtered by access rules)
`/catalogue`	Equipment categories and tiers	All signed-in users
`/notifications`, `/profile`	Notices; HR profile and delegation	All signed-in users
`/approvals`	Manager approval queue	`manager` role
`/reviews`, `/standards`	CICT review queue; specification profile maintenance	isCict
`/procurement`	Procurement queue	`procurement` (and admin)
`/stores`	Goods receipt queue	`stores` (and admin)
`/verification`	Verification queue	`cict_support` (and admin)

Route	Page	Shown to
`/reports`, `/directory`	Reports; staff directory	isOversight
`/audit`	Audit workspace	`auditor` (and admin)
`/admin`	Administration (8 tabs)	`admin`

Restricted pages also check the role on the page itself and show a "Restricted" message if opened directly by URL; the API enforces the same rules regardless.

10.3 Conventions for contributors

- Fetch with React Query (`useQuery`) and invalidate the relevant query keys after a mutation so lists refresh.
- Show API errors with `toast.error(err instanceof ApiError ? err.message : "…")`; show load failures with `QueryError` and a Retry button.
- Run `npx tsc --noEmit` and `npx eslint src` before committing. Prettier formatting is enforced through ESLint (`npx eslint src --fix`).
- The project is connected to Lovable: commits pushed to the connected branch sync back to the Lovable editor. Never force-push or rewrite pushed history.

11. Configuration

The backend reads `backend/src/main/resources/application.yml`; every sensitive or environment-specific value can be overridden with an environment variable.

11.1 Backend settings

Setting	Environment variable	Default	Purpose
`server.port`	`SERVER_PORT`	`8080`	HTTP port.
`spring.datasource.url`	`SPRING_DATASOURCE_URL`	`jdbc:postgresql://localhost:5432/specification_db`	Database connection.
`spring.datasource.username` / `password`	`SPRING_DATASOURCE_USERNAME` / `_PASSWORD`	`postgres` / (development value)	Database credentials.
`spring.servlet.multipart.max-file-size`	—	`25MB`	Upload size limit.
`app.jwt.secret`	`JWT_SECRET`	public development value	Token signing key (≥ 32 bytes). **Must be changed.**
`app.jwt.expiration-minutes`	`APP_JWT_EXPIRATIONMINUTES`	`480`	Session length.
`app.cors.allowed-origins`	`APP_CORS_ALLOWEDORIGINS`	localhost dev ports	Frontend origins allowed to call the API.
`app.storage.attachments-dir`	`ATTACHMENTS_DIR`	`./data/attachments`	Where uploaded files are written.
`app.hr.login-url`	`HR_LOGIN_URL`	`https://hr.unza.zm/api/auth-login2`	HR sign-in endpoint.
`app.hr.metadata-url`	`HR_METADATA_URL`	`https://hr.unza.zm/api/get-meta-data-on-user`	HR profile endpoint.
`app.mail.enabled`	`MAIL_ENABLED`	`false`	Send email copies of notifications.
`app.mail.from`	`MAIL_FROM`	`no-reply@unza.zm`	Sender address.
`spring.mail.*`	`SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`	unset / `587`	SMTP server (STARTTLS).

11.2 Frontend settings

Variable	Purpose
`VITE_API_BASE_URL`	Base URL of the backend, for example `https://spec-api.unza.zm`. Defaults to `http://localhost:8080`. Set in `.env` for local work and in the hosting environment for builds.

11.3 In-application settings

Organisation name, unit name, support email, declaration text and SLA days are stored in `app_settings` and edited by administrators in **Administration → Settings** without a redeploy (Appendix C). Drop-down lists are edited in **Administration → Lists** (Appendix A).

12. Building, running and deploying

12.1 Prerequisites

Tool	Version	Used for
Java JDK	17 or later (tested on 21)	Backend build and run
Apache Maven	3.9+	Backend build and tests
PostgreSQL	16 or later	Database (and the local test database)
Node.js	22+ (tested on 24)	Frontend build
npm or Bun	current	Frontend dependencies
Docker (optional)	current	Lets tests use a disposable Postgres container

12.2 Run locally

1. Database

```
psql -U postgres -c "CREATE DATABASE specification_db;"
```

No manual schema work is needed: Flyway creates and updates the schema when the backend starts.

2. Backend

```
cd backend
mvn spring-boot:run            # starts on http://localhost:8080
# health check:  http://localhost:8080/actuator/health
# API docs:      http://localhost:8080/swagger-ui.html
```

3. Frontend

```
npm install
echo VITE_API_BASE_URL="http://localhost:8080" > .env
npm run dev                    # prints the local URL
```

> **Good to know.** Sign-in always goes through the live UNZA HR API, so the backend machine needs network access to `hr.unza.zm` and you need valid staff credentials. To work offline, an administrator can provision accounts in Administration → Staff accounts, but those users still sign in through HR.

12.3 Production build

Backend

```
cd backend
mvn clean package              # runs tests, produces target/specification-backend-0.0.1-SNAPSHOT.
jar
java -jar target/specification-backend-0.0.1-SNAPSHOT.jar
```

Run the jar as a Windows service or systemd unit, behind a reverse proxy (IIS, Nginx) that terminates HTTPS. Provide the configuration through environment variables (Section 11).

Frontend

```
npm run build                  # output in .output/ (Nitro, Cloudflare Workers target)
npx vite preview               # preview the production build locally
```

The default build target is Cloudflare Workers (configuration is generated in `.output/server/wrangler.json`). The project can also be published from the Lovable editor. Set `VITE_API_BASE_URL` to the production API URL before building, and add the frontend's production origin to the backend's `APP_CORS_ALLOWEDORIGINS`.

12.4 Production checklist

	Item
1	Set `JWT_SECRET` to a long random value; confirm the start-up warning no longer appears.
2	Move the database password out of `application.yml` into `SPRING_DATASOURCE_PASSWORD`; use a dedicated database user rather than `postgres`.
3	Set `APP_CORS_ALLOWEDORIGINS` to the production frontend origin only.
4	Serve both tiers over HTTPS.
5	Point `ATTACHMENTS_DIR` at durable storage that is included in backups.
6	Schedule the backup script daily and test a restore (Section 14).
7	Decide on email: configure SMTP and set `MAIL_ENABLED=true`, or leave in-app only.
8	Consider restricting `/swagger-ui` and `/v3/api-docs` to the CICT network.
9	Assign the first administrator role directly in the database for the initial CICT administrator, then manage roles in the UI.

Granting the first administrator (run once, after that person has signed in at least once):

```
INSERT INTO user_roles (id, user_id, role)
SELECT gen_random_uuid(), id, 'admin' FROM users WHERE email = 'firstname.lastname@unza.zm';
```

13. Testing

The backend has 15 integration tests that boot the full application against a real PostgreSQL database and drive it through the HTTP API, with only the HR system mocked.

13.1 Test suites

Class	Tests	What it proves
`AuthIntegrationTest`	3	Valid HR credentials issue a token; rejected credentials return 502; repeated failures lock the identifier out with 429.
`RequestLifecycleIntegrationTest`	4	Reference numbers are generated and unique; negative quantities are rejected; a request goes draft → manager → CICT → approved; a non-CICT user cannot approve the technical review.
`ProcurementPipelineIntegrationTest`	4	Happy path to `completed` with sequential EVAL/VERIFY references; failing evaluation needs a reason; a failed quotation cannot be purchased; needs-return loops back via restart.
`NotificationIntegrationTest`	4	Each stage notifies the next person and never the actor; clarification and rejection reach the requester; an active delegate is notified and can approve; only the owner can read or dismiss a notification.

13.2 Running the tests

```
cd backend
mvn test
```

`AbstractIntegrationTest` chooses the database automatically:

- **Docker available:** a disposable `postgres:16-alpine` container is started for the run.
- **No Docker:** a local database named `specification_test` is dropped and recreated at the start of the run, then migrated by Flyway. The code refuses to reset any database whose name does not end in `_test`, so the application database can never be wiped by tests.

Variable	Default	Purpose
`TEST_DB_HOST`	`localhost`	Local test database host
`TEST_DB_PORT`	`5432`	Port
`TEST_DB_NAME`	`specification_test`	Database name (must end in `_test`)
`TEST_DB_USERNAME` / `TEST_DB_PASSWORD`	`postgres` / development value	Credentials with permission to create databases

When Docker is not running, Testcontainers logs one "Could not find a valid Docker environment" error line before the fallback starts. This is expected and is not a test failure.

13.3 Frontend checks

```
npx tsc --noEmit      # type checking
npx eslint src        # lint + Prettier formatting rules
npm run build         # production build
```

14. Operations

14.1 Backup and restore

Two PowerShell scripts in `backend/scripts/` need only the PostgreSQL client tools.

```
cd backend/scripts
./backup.ps1                                        # back up now
./restore.ps1                                       # dry run: shows what would happen
./restore.ps1 -BackupPath 20260811-020000 -Force    # actually restore
```

- Each backup writes `backend/backups/<yyyyMMdd-HHmmss>/` with `database.dump` (pg_dump custom format), `attachments.zip` and `manifest.json`. The 14 most recent backups are kept.
- Restore is destructive and always a dry run unless `-Force` is given. Stop the backend before restoring.
- Backups are excluded from git. Copy them to durable storage (another disk, network share or cloud), and test a restore regularly.

Schedule a daily backup with Windows Task Scheduler:

```
schtasks /create /tn "UNZA Spec System Backup" /sc daily /st 02:00 ^
  /tr "powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\path\to\backend\scripts\backup.p
s1"
```

14.2 Monitoring and logs

- **Health:** `GET /actuator/health` returns `{"status":"UP"}` when the application and database are reachable.
- **Logs:** standard Spring Boot console output at INFO level for the application package. Unhandled errors are logged with a stack trace; data-integrity conflicts are logged as warnings.
- **Audit:** administrative actions are visible to auditors and administrators in the Audit log screens and can be exported to CSV.

14.3 Troubleshooting

Symptom	Likely cause	What to do
Sign-in says the HR system could not be reached	No network route to `hr.unza.zm`, or HR is down	Check connectivity from the backend host; check the HR URLs in configuration.
"Your staff account is not active"	HR status is not 1	The person must contact HR; the system cannot override this.
"Too many sign-in attempts"	5 failed attempts	Wait 15 minutes, or restart the backend to clear the in-memory counter.
Frontend shows "Could not reach the server"	Wrong `VITE_API_BASE_URL`, backend down, or CORS rejection	Check the API URL, the health endpoint, and that the frontend origin is in the allowed origins list.
Backend fails at start-up with a Flyway or validation error	Schema differs from the entities, or a migration was edited after being applied	Never edit applied migrations; add a new `V17__...` script. Restore from backup if needed.
Downloads fail with "Attachment file is missing on disk"	Attachments directory moved or not restored	Restore `attachments.zip` from the matching backup into `ATTACHMENTS_DIR`.

Symptom	Likely cause	What to do
A user lacks a menu item they need	Role not assigned	Assign it in Administration → Staff roles; it takes effect on their next action.
Cannot delete a category or profile	Still referenced by requests	Retire it instead; this is intended (Section 8.3).

14.4 Making schema changes

1. Add a new migration file `backend/src/main/resources/db/migration/V17__short_description.sql` (next number in sequence).
2. Update the JPA entity in `domain/` to match; Hibernate validates the mapping at start-up.
3. Run `mvn test`; the tests migrate a fresh database from V1, which proves the migration works from scratch.
4. Never change a migration that has already run in any shared environment.

15. Known limitations and recommendations

Area	Current state	Recommendation
JWT secret	Default secret published in source.	Set `JWT_SECRET` in every deployment (essential).
Database credentials	Development password stored in `application.yml`.	Supply through environment variables; use a least-privilege database user.
Sign-in rate limit	In memory, per instance.	Move to a shared store if several backend instances are run.
Email	Disabled by default.	Configure SMTP if email copies are wanted.
Attachments	Stored on the backend server's local disk.	Use durable or network storage and include it in backups.
Token storage	Token held by the browser; valid for 8 hours; no server-side revocation.	Shorten expiry if needed; role changes already apply immediately.
API documentation	Swagger UI is public.	Restrict to the CICT network in production.
HR dependency	No HR means no sign-in.	Monitor HR availability; agree an escalation path with HR.

Appendix A — Lookup lists

Values seeded in `lookup_values` and editable in Administration → Lists. Values as at 13 October 2026.

List key	Values
`asset_class`	End-user computing, Peripheral, Networking, Server and datacentre, Audio visual, Power and protection, Software
`budget_status`	Budgeted for current year, Not yet budgeted, Funded by project or grant, Unknown
`business_context`	Administration, Teaching, Research, Laboratory, Student service, Grant / project
`campus`	Great East Road Campus, Ridgeway Campus, Liempe Farm
`designation`	31 designations from Vice-Chancellor to Support Staff, plus "Other (please specify)"
`position_level`	Support staff, Officer, Senior Officer, Principal Officer, Technician, Assistant Lecturer, Lecturer, Senior Lecturer, Principal Lecturer, Researcher, Specialist, Manager, Assistant Director, Deputy Director, Associate Professor, Professor, Head of Department, Deputy Dean, Dean, Registrar, Bursar, Director, Top management, Executive management
`priority`	Normal, High, Urgent
`request_type`	New requirement, Replacement, Upgrade, Expansion, Shared resource, Project requirement, Temporary need
`role`	Display labels for the nine roles
`software_category`	Design, Teaching, Development, Other
`spec_tier`	Standard, Enhanced, Specialised

Appendix B — Equipment categories

Active categories in the catalogue (19). Codes `PRINTER` and the `Software` asset class have extra validation rules (Section 5.3). The catalogue also holds 29 active specification profiles across the three tiers.

Code	Name	Asset class
`LAPTOP`	Laptop	End-user computing
`DESKTOP`	Desktop Computer	End-user computing
`APPLE`	Apple Computer	End-user computing
`TABLET`	Tablet	End-user computing
`PRINTER`	Printer	Peripheral
`PROJECTOR`	Projector	Audio visual
`SCANNER`	Scanner	Peripheral
`NETWORK_SWITCH`	Network Switch	Networking
`NETWORK_ROUTER`	Router	Networking
`NETWORK_CABLE`	Network Cable	Networking
`NETWORK_TRUNKING`	Cable Trunking / Management	Networking
`NETWORK_ACCESS_POINT`	Wireless Access Point	Networking
`SOFTWARE_LICENCE`	Software Licence	Software
`MONITOR`	Monitor	Peripheral
`UPS`	Uninterruptible Power Supply (UPS)	Power and protection
`POWER_SURGE_PROTECTOR`	Power Adaptor / Surge Protector	Power and protection
`SERVER`	Server	Server and datacentre
`NETWORK_ACCESSORY`	Network Rack / Patch Panel / Connector	Networking
`ICT_ACCESSORY`	ICT Accessory / Other Equipment	Peripheral

Appendix C — System settings

Key	Default value	Used for
`organisation_name`	University of Zambia	Header and document letterhead
`unit_name`	CICT · Equipment Specifications	Subtitle beside the organisation name
`support_email`	servicedesk@unza.zm	Contact link on sign-in and help pages
`declaration_text`	I confirm that the information provided is accurate…	Declaration the requester must accept
`manager_sla_days`	3	Working days a manager has to act (shown to users)
`cict_sla_days`	5	Working days for CICT technical review (shown to users)

Appendix D — Database migrations

Version	Purpose
V1	Initial schema.
V2	Seed data.
V3	Real equipment catalogue (replaces seed catalogue).
V4	Remove expired MacBook specifications.
V5	Year-scoped request references.
V6	System audit log.
V7	Procurement pipeline tables (quotations, evaluations, purchases, receipts, verifications).
V8	Restore application settings removed by V3.
V9	Restore lookup values.
V10	Request department, new reference numbers and catalogue updates.
V11	Item number of users; evaluable catalogue fields.
V12	Designation of intended user.
V13	Expanded designation list.
V14	Per-item designation.
V15	Allow hard deletion of users.
V16	Quotation attachment (vendor quote document).